Yggdrasil ERP — Desktop Client Architecture

Overview

The Yggdrasil ERP desktop client is the reference user interface implementation for the platform. It is a native application built with C++17 and Qt 6, combining QML for presentation with C++ service layers for networking, authentication, and application state.

Unlike browser-first ERP systems, Yggdrasil follows a **desktop-first development model**. All client features are implemented and validated in the desktop client before being reproduced in the web interface. This ensures that the platform remains suitable for operational environments such as manufacturing floors, warehouses, and service operations where browser constraints can become limiting.

The desktop client communicates with the Yggdrasil server exclusively through the public REST and WebSocket interfaces, making it both a production client and a functional reference implementation for third-party integrations.

Technology Foundation

Component	Implementation
Language	C++17
UI Framework	Qt 6 (QML + Qt Quick Controls)
Charting	Qt Charts
Build System	CMake
Networking	REST over HTTP + WebSocket
Target Platforms	Windows, Linux, macOS

The decision to build the client in native Qt rather than as a browser application provides several operational advantages:

• consistent performance with large datasets • reliable multi-window workflows • direct hardware integration (barcode scanners, printers, measurement devices) • deterministic UI behavior without browser lifecycle constraints

These properties are particularly valuable in industrial environments where operators interact with ERP systems continuously throughout the workday.

Application Architecture

Initialization Sequence

Application startup follows a deterministic sequence:

1. Initialize `QGuiApplication`
2. Create the QML engine
3. Register backend service singletons
4. Load compiled QML resources
5. Create the main application window
6. Perform authentication check

Depending on authentication state, the client either displays the login dialog or loads the primary navigation interface.

Backend Service Layer

The desktop client exposes several C++ backend services to the QML layer through global singleton objects. These services handle networking, authentication, configuration, and application-wide state.

Service	Responsibility
ApiClient	HTTP communication with the server API
AuthManager	login, logout, MFA handling, token lifecycle
WebSocketManager	real-time event subscription
ThemeManager	centralized visual styling tokens
SettingsManager	user preferences and connection settings
NotificationManager	in-app notifications and alerts

These services form the boundary between presentation logic and operational functionality. QML components interact only with these services and never communicate directly with network layers.

QML Interface Architecture

The user interface is composed of module pages that correspond directly to server business domains.

Major modules include:

• Dashboard • CRM • Sales • Purchasing • Manufacturing • Warehouse • Finance • Projects • PLM • Quality • Service • HR • Logistics • Forms

Each module page acts as a container for data tables, detail views, and workflow actions.

Reusable QML components provide consistent interaction patterns across modules.

Key components include:

• DataTable — paginated searchable grid • RecordDetailPage — master/detail entity view • FormDialog — create/edit forms • AttachmentPanel — document management • NotificationPanel — real-time alerts • ChartCard — dashboard visualization container

This component approach ensures interface consistency across the entire ERP surface.

Data Flow Model

All communication between the desktop client and the platform occurs through the server API.

UI interaction follows a predictable pattern:

QML Interface → ApiClient → HTTP Request → Server → HTTP Response → UI Update

Server-generated events are delivered via WebSocket.

Server Event → WebSocketManager → Qt Signal → UI Component

This model eliminates polling and enables immediate UI updates when data changes elsewhere in the system.

Authentication Workflow

Authentication is managed by the `AuthManager` service.

Login sequence:

1. User submits credentials
2. Client calls `/api/auth/login`
3. If MFA is required, a TOTP challenge is presented
4. Access and refresh tokens are stored securely
5. Main application interface loads

Access tokens are refreshed automatically when approaching expiration. WebSocket connections re-authenticate when token refresh occurs.

The client also supports **operator login** for shared workstations, allowing rapid user switching without restarting the application.

Desktop-First Development Model

Feature development across the platform follows a strict sequence:

1. Define server API and data structures
2. Implement backend functionality
3. Build the desktop interface
4. Port the feature to the web client

This approach ensures that the most capable interface is always the primary implementation. The web client becomes a portability layer rather than the architectural reference.

Theming System

The application uses a centralized theme manager providing design tokens for:

• colors • typography • spacing • component elevation • border radii

QML components reference theme properties rather than hardcoded values, allowing global visual changes without rewriting interface code.

Workflow Actions

ERP operations frequently require entity-specific actions. The desktop client exposes these through contextual action buttons on entity detail pages.

Examples include:

• converting quotes to orders • generating invoices • completing work order units • creating purchase bills

These actions correspond directly to server endpoints and represent operational transitions in the system's workflow state machine.

Build and Distribution

The client is built using CMake and the Qt toolchain.

Typical build procedure:

```
mkdir build
cd build
cmake ..
make
```

Supported platforms:

Platform	Status
Windows	primary development
Linux	production environments
macOS	supported

Qt resource compilation embeds QML files and assets directly into the binary, eliminating runtime file dependency issues.

Performance Characteristics

Typical performance profile:

• application launch to login screen: <2 seconds • login to dashboard: ~3 seconds on local network • memory footprint: ~80–120 MB base

Large datasets are handled through server-side pagination and virtualized table rendering. Real-time updates are delivered through WebSocket events rather than polling.

Architectural Role

Within the broader Yggdrasil architecture, the desktop client serves three roles simultaneously:

1. Primary operational interface for enterprise users
2. Reference implementation for the public API
3. Validation environment for new platform features

Because it interacts with the server exclusively through public interfaces, the client doubles as a continuously maintained example of correct API usage.

This approach encourages architectural discipline and ensures that platform capabilities remain usable by external integrations as well as internal clients.